Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

Cómo programar en LatinChain Code (Lenguaje de Programación hecho con IA)

Lenguaje de Programación en Español

Programming books

Written and produced by Lcdo. César Cordero Rodríguez

More info here:

Personal Blog: https://dev-rockcesar.blogspot.com

Portfolio: https://rockcesar.github.io

Author LinkedIn:

https://www.linkedin.com/in/c%C3%A9sar-cordero-34556186/

Democratizing the Software!

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

Author Biography

Bachelor's Degree in Computer Science, e-Business Applications Specialist, Networking Specialist, IBM training certified, CISCO training certified, Odoo Technical Training, Odoo Functional Training. AWS Architect. Quantum Computing Developer. Software Engineer in Dockers, Qiskit, C, C++, Java, JavaScript, PHP, Python, C#, Mono Develop, Ruby, Solidity, OpenERP 7, Odoo 8, 9, 10, 11, 12, 13, 14, 15, 16 and 17, Django, Flask, Wordpress, Joomla, Symfony2, CodeIgniter, OpenCV, jQuery, AngularJS, QT, AureliaJS and NodeJS. GNU/Linux, MacOS and Windows. I'm Free Software and OpenSource Expert.

21 years of experience in informatic field, 19 years in GNU/Linux, 21 years in C, 21 years in C++, 19 years in Java, 15 years in PHP, 15 years in JavaScript, 11 years in Python and 11 years of experience in Odoo (OpenERP 7, Odoo 8, 9, 10, 11, 12, 13, 14, 15, 16 and 17).

Bilingual: English and Spanish.

Portfolio:

https://rockcesar.github.io

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

Summary

Summary 3
Guía Completa de Programación en LatinChain Code 5
1. Primeros Pasos: Ejecutar y Compilar 5
2. Sintaxis Básica, Comentarios y UTF-8 5
3. Imprimir en Pantalla 6
4. Variables y Tipos de Datos 6
Reasignación de Variables 7
5. Operadores 7
Operadores Aritméticos 7
Operadores de Comparación 8
6. Estructuras de Control: Condicionales (si) 8
7. Estructuras de Control: Ciclos (mientras) 8
8. Sistema de Módulos e Importaciones 9
Importar un archivo en la misma carpeta 9
Importar usando rutas relativas 10
Importar usando rutas absolutas 10
Importar todos los módulos de un directorio (Comodines y Prefijos *) 10
9. Programa de Ejemplo Completo 11
Acerca de LatinChain Code 12

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

This writing is under GNU/Free Documentation License, since October 15, 2024.

Lcdo. César Cordero Rodríguez

Programming book, written by César Cordero Rodríguez!

Title: Cómo programar en LatinChain Code (Lenguaje de Programación hecho con IA)

Author: César Cordero Rodríguez

Year: 2026

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

Guía Completa de Programación en LatinChain Code

¡Bienvenido a **LatinChain Code**! Un lenguaje de programación multiplataforma, de código abierto (GPLv3) y con sintaxis en español, diseñado por el **Lcdo. César Cordero**.

Este lenguaje está diseñado para ser fácil de aprender, intuitivo e ideal para hispanohablantes. Permite tanto la interpretación directa de código fuente como la compilación a ejecutables (`.lac`).

1. Primeros Pasos: Ejecutar y Compilar

Antes de programar, necesitas saber cómo ejecutar tu código. Guarda tu código fuente en un archivo con la extensión **`.la`** (por ejemplo, `programa.la`).

Abre tu terminal o línea de comandos y utiliza el motor de LatinChain Code (`latinchain.py`):

Interpretar al vuelo: Ejecuta el código directamente sin generar binarios.
python latinchain.py run programa.la

-

Compilar: Genera un archivo ejecutable/bytecode (creará `programa.lac`). Al compilar, todos los módulos que importes se incluirán en este único archivo.
python latinchain.py compile programa.la

-

Ejecutar binario: Ejecuta el archivo compilado previamente.
python latinchain.py exec programa.lac

-

2. Sintaxis Básica, Comentarios y UTF-8

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

Todo programa necesita documentarse. En LatinChain Code, los comentarios de una sola línea se escriben utilizando dos barras diagonales `//`.

```
// Este es un comentario en LatinChain Code
// El motor ignorará esta línea
```

Soporte UTF-8 (Caracteres en Español): El lenguaje soporta nativamente caracteres especiales del español. Puedes usar acentos y la letra eñe (ñ) tanto en textos como en los nombres de tus variables y módulos.

Nota: Toda declaración de código (asignación de variables, impresión, etc.) debe terminar con un punto y coma `;` (excepto la palabra reservada `importar`, donde es opcional).

3. Imprimir en Pantalla

Para mostrar mensajes, resultados o valores en la consola, utilizamos la palabra reservada `imprimir()`.

```
imprimir("¡Hola, mundo desde LatinChain!");
imprimir("El año actual es:");
imprimir(2026);
```

4. Variables y Tipos de Datos

Para declarar una variable, se utiliza la palabra clave `variable`. LatinChain Code infiere dinámicamente el tipo de dato que le asignas.

Soporta tres tipos de datos básicos: **Números**, **Cadenas de texto** (Strings) y **Booleanos** (`verdadero` o `falso`).

```
// 1. Números (Enteros y Decimales)
variable límite = 35; // Observa el uso de la tilde
variable pi = 3.1416;
```

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

```
// 2. Cadenas de Texto
variable nombre = "César Cordero";
variable año_creación = "2026"; // Uso de la ñ y acentos

// 3. Valores Booleanos
variable es_programador = verdadero;
variable es_aburrido = falso;

// Mostrando las variables
imprimir(nombre);
imprimir(límite);
```

Reasignación de Variables

Puedes cambiar el valor de una variable después de crearla, sin usar la palabra `variable` de nuevo:

```
variable puntos = 10;
puntos = 20; // Reasignación
puntos = puntos + 5; // Ahora puntos vale 25
```

5. Operadores

Operadores Aritméticos

Puedes realizar operaciones matemáticas básicas:

- Suma: `+`
- Resta: `-`
- Multiplicación: `*`
- División: `/`

```
variable resultado = (10 + 5) * 2; // resultado es 30
```

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

Operadores de Comparación

Se usan para comparar valores, devolviendo `verdadero` o `falso`:

- Igual a: `==`
- Diferente de: `!=`
- Mayor que: `>`
- Menor que: `<`
- Mayor o igual que: `>=`
- Menor o igual que: `<=`

6. Estructuras de Control: Condicionales (si)

La estructura `si` evalúa una condición. Si la condición es `verdadero`, se ejecuta el bloque de código dentro de las llaves `{ }`.

```
variable saldo = 100;
variable costo = 40;

si (saldo >= costo) {
   imprimir("Puedes realizar la compra.");
   saldo = saldo - costo;
}

si (saldo < costo) {
   imprimir("Saldo insuficiente.");
}
```

7. Estructuras de Control: Ciclos (mientras)

Para repetir una acción varias veces, usamos el bucle `mientras`. Este bloque se repetirá una y otra vez mientras la condición sea `verdadero`.

```
variable contador = 1;
```

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

```
variable límite = 5;

imprimir("Iniciando el conteo:");

mientras (contador <= límite) {
   imprimir(contador);
   contador = contador + 1; // Importante para no hacer un bucle infinito
}

imprimir("Conteo finalizado.");
```

8. Sistema de Módulos e Importaciones

LatinChain Code te permite dividir proyectos grandes en múltiples archivos y carpetas para mantener todo organizado.

Para usar código de otro archivo, utiliza la palabra `importar` seguida del nombre del archivo o ruta (sin la extensión `.la`). El lenguaje soporta la importación a través de nombres simples, rutas relativas y rutas absolutas.

Importar un archivo en la misma carpeta

Si tienes un archivo llamado `matematicas.la` en el mismo directorio, simplemente usa su nombre:

```
// matematicas.la
variable pi = 3.1416;
imprimir("Módulo de matemáticas listo.");
```

Puedes usarlo en tu programa principal así:

```
// principal.la
importar matematicas

imprimir("El valor de PI es:");
```

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

imprimir(pi);

Importar usando rutas relativas

Las rutas relativas toman como punto de partida la ubicación del archivo en el que te encuentras escribiendo. Puedes usar `./` para hacer referencia a la carpeta actual, o `../` para subir a un directorio superior:

```
// Cargar un módulo que está en la misma carpeta
importar ./maths

// Cargar un módulo que está en un directorio superior
importar ../matematicas/maths

// Cargar un módulo dentro de una subcarpeta
importar utilidades/textos
```

Importar usando rutas absolutas

Si necesitas indicar la ubicación exacta de un módulo desde la raíz de tu sistema operativo (útil en servidores Linux/macOS o estructuras complejas), puedes proporcionar la ruta completa:

```
// Asumiendo que el código principal está en /home/usuario1/matematicas/principal.la
importar /home/usuario1/matematicas/maths
importar /opt/latinchain/librerias/core
```

Importar todos los módulos de un directorio (Comodines y Prefijos `*`)

Si deseas importar automáticamente múltiples archivos `.la` sin tener que nombrarlos uno por uno, puedes utilizar el comodín `*`. Si lo acompañas de una palabra (prefijo), solo importará los archivos que comiencen con esa palabra:

```
// Importar todos los archivos .la de la carpeta actual
```

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

```
importar ./*

// Importar todos los archivos .la de un directorio superior
importar ../matematicas/*

// Importar SOLO los archivos que comiencen con "maths" en el directorio actual
// (ej. maths1.la, maths2.la, maths_avanzadas.la)
importar ./maths*

// Importar por prefijo desde una ruta absoluta
importar /home/usuario1/utilidades/core_*
```

9. Programa de Ejemplo Completo

Aquí tienes un script completo que junta todo lo que hemos aprendido. Puedes guardarlo como `ejemplo.la` y probarlo.

```
// Programa: Adivinador Básico
// Escrito en LatinChain Code

importar ./utilidades/bienvenida // Suponiendo que tienes este módulo en una subcarpeta

variable número_secreto = 7;
variable intento = 1;
variable acierto = falso;

imprimir("--- BIENVENIDO AL JUEGO EN LATINCHAIN ---");
imprimir("Evaluando el número secreto...");

// Simulamos los intentos probando números del 1 al 10
mientras (intento <= 10) {

    si (intento == número_secreto) {
```

Lcdo. César Cordero Rodríguez
Pi Hackathon winner 2021
Telegram: t.me/rockcesar
Email: cesar.cordero.r@gmail.com

```
        imprimir("¡Éxito! El número secreto es:");
        imprimir(intento);
        acierto = verdadero;
    }

    intento = intento + 1;
}

si (acierto == verdadero) {
    imprimir("Fin del programa. ¡Gracias por usar LatinChain Code!");
}
```

Acerca de LatinChain Code

- **Diseño:** Intuitivo, con palabras reservadas en español y soporte total de caracteres especiales (UTF-8).
- **Organización:** Soporte modular mediante importaciones relativas, absolutas, comodines y prefijos (*).
- **Paradigma:** Imperativo, tipado dinámico.
- **Licencia:** GNU GPLv3.
- **Creador:** Lcdo. César Cordero.